exp02_selection_scale 实验报告

动态任务分配两阶段框架：任务选择层的规模压缩收益与标定口径

实验日期：2026–10–20　随机种子：seed = 42　扫描矩阵：3 规模 × 5 变体 + 全量对照

摘要

本报告记录 exp02_selection_scale 实验的设置与结果。实验针对 Chapter3_README §1(4) 的待验证命题——当 $|T_t| \gg K$ 时，第一阶段「动态任务选择」的候选压缩（$n_{\text{cand}} = \lceil \alpha K \rceil$）能否提高实时性——在真实 DEM 地形（SRTM/ASTER 级高程数据，200 m 采样）上固定 $K = 8$、满电航程 80 km，将任务规模扫到 50/100/200，并对两种规模标定口径（机数名额族 `quota_*` vs 机队能力驱动族 `cap_x*`）做 17 格对照扫描。结果显示：(1) 压缩真实发生且与 N **解耦**——候选数恒为 20/40 或 $\text{cap}(t) = 10/20/40$，不随任务规模变化；(2) 第二阶段耗时与 N **无关**（固定 $n_{\text{sel}} = 16$ 时 25.3/25.2/25.0 s，三次测量极差 $< 2\%$）；(3) 原结论「α 需按机队总吞吐标定」被修正为更精确的答案——intersect 口径下**真正瓶颈是选择数** $n_{\text{sel}} = \lceil \text{max_ratio} \cdot K \rceil = 16$ **而非** α（`quota_a2.5` 与 `quota_a5` 结果几乎相同）；(4) 能力驱动口径（选择数接到 $\text{cap}(t)$）把压缩损失大部分买回来：$n_{\text{sel}} \approx \text{cap}(t)$ 时 F1 达全量的 93.5%（N=50）/ 83.4%（N=100），而耗时仅为其 87%/13%（N=100 加速 7.6×），$\text{cap}(t)$ 即「质量—耗时」的天然平衡点。实验同时验证了代价矩阵延迟构建（惰性表）的效果：N=200 时一阶段仅现算 2,164 条边（全量的 5.0%），端到端 $O(n^2)$ 构建占比从旧版 85.6% 降到 3.7%。

目录

1 实验设置 **3**

1.1 实验目标与评价口径 3

1.2 实验定位与证据边界 3

1.3 场景与地形 4

1.4 变体矩阵：两种规模标定口径 4

1.5 求解预算与计时口径 4

2 主要结果 **5**

2.1 实测总表 5

2.2 结论 6

2.3 已知问题与边界 6

3 图表 **7**

3.1 吞吐—耗时—质量权衡（核心图） 7

3.2 规模压缩与 N 解耦（成功标准 1） 7

3.3 耗时分解与惰性边（成功标准 2 + 优化 A 证据） 8

3.4 瓶颈定位：选择数封顶而非候选倍率 8

3.5 能力驱动标定：cap(t) 是天然平衡点（成功标准 3/4） 9

4 附：数据与复现 **9**

1 实验设置

1.1 实验目标与评价口径

本实验验证两阶段分层框架中**第一阶段（动态任务选择）的规模压缩机制**，对应论文 Chapter3_README §1(4) 与 §3.3。待验证命题：

> 当 $|T_t| \gg K(t)$ 时，把全部任务直接交给精细组合优化会显著扩大搜索空间；第一阶段用 $n_{\text{cand}} = \lceil \alpha K \rceil$ 把 T_t 压到 T_t^{sel}，据称能提高实时性。

`exp01_smoke` 跑在「机多任务少」一侧（10 任务 / 24 机，$\lceil \alpha K \rceil = 60$），$n_{\text{cand}}$ 截断从不触发，无法验证该机制。本实验固定 $K = 8$（8 巢 × 容量 4），把任务规模拉到 50/100/200，测量四个量：

- T_{hint}：`cost_hint`（最近可用机，惰性表 $O(n \cdot K)$ 条边）；
- T_{sel}：第一阶段耗时 + 压缩率 $|T_t| \to |T_t^{\text{sel}}|$；
- T_{build}：候选子集三维矩阵构建（与一阶段共享惰性表缓存）；
- T_{muas}：第二阶段对候选集求解的耗时。

由此回答三个问题：(1) 压缩是否真的发生了？压到多少？(2) 分层省下的搜索时间，是否被 $O(n^2)$ 代价构建吃掉？(3) 候选规模按「机数名额」还是「机队能力」标定，吞吐—耗时—质量的平衡点在哪里？

指标口径（2026-10-22 v2）：报告层的完成/按时/成本类指标一律采用论文主表 v2 口径——最终完成率 η_{final}（分母排除物理不可达、淘汰须为 0）、按时率（最终交付口径，迟到计完成不计按时）、单位完成航程 = 总航程（含返航）/完成数；过程指标含名额利用率 $\sum n_{\text{sel}}^{\text{used}} / \sum n_{\text{sel}}^{\text{cap}}$ 与跨周期推迟量，方法诊断含 n_{skip} / 一阶 shortfall 分账（定义详见 `exp01_smoke/report.tex` §1.2）。

1.2 实验定位与证据边界

本实验属 **L2 设计有效性**证据：验证「选择层压缩」这一设计在 $|T_t| \gg K$ 规模下真实有效、且给出标定口径。本报告**可以**声称：压缩机制按协议工作（四项目标标准全部达成，见 §2.2）、两种标定口径的权衡曲线可定位、$O(n^2)$ 构建瓶颈已被延迟构建消除。本报告**不能**声称：

- **方法优于任何基线**——对比方法尚未接入统一评价（属 exp05 职责）；本实验的「全量对照」（不经过第一阶段直接全量求解）是**同一求解器**的质量上界参照，不是方法学基线；
- **单 seed 稳健性**——仅 seed 42；多种子稳健性未做（exp01 侧已做 3 seed）；
- **生产级解质量**——求解预算为 40 代 + 800 评估（exp01 生产为 100 代 + 2000），本实验量的是权衡曲线**形状**，各格同预算、横向可比；
- **N=200 的质量上界**——全量对照单格求解约 25 min、代价不成比例，未测；该规模的「无压缩上界」由 N=50/100 的损失比例外推说明（机制不随 N 变，结论 2 已证明耗时与 N 无关）。

1.3 场景与地形

复用 exp01_smoke 的场景口径：实验区域位于青藏高原（约 N29°/E091°），高程数据取自 `ASTGTMV003_N29E091_dem.tif`，采样步长 200 m，8 个机巢，场景范围约 101.6×115.0 km。

距离口径：所有距离一律采用**垂直切面 DEM 三维航程**，凡由距离推导时间（ETA、时间窗判定、航程约束、边际插入估计）均由统一的代价提供者计算，保证全链路口径一致。

机群构型（2026-10-20 修复项）：无人机按 exp01 `build_uavs` 口径 **驻留其机巢**（UAV $i \to$ 巢 $i \bmod 8$），满电航程 80 km。旧版把全部 UAV 放在局部平面原点，最近可用机距离中位数被推到 39.4 km、cap(t) 缩到 16、24/50 个任务被往返预筛误杀——整个规模实验测的是「原点邻域」这个退化构型。驻巢修复后：中位距离 15.2 km、cap(t) ≈ 41、预筛零误杀，与全量对照的执行数同量级。

1.4 变体矩阵：两种规模标定口径

扫描矩阵固定任务生成参数（锚定机巢撒点、spread 25 km、奖励 $1 \sim 10$、时间窗 $1 \sim 8$ h），只变「候选/选择规模政策」：

表 1：变体矩阵（两个标定族 + 全量对照）

变体	族	α	capability 模式	含义
`quota_a2.5`	名额族	2.5	intersect（收紧）	$n_{\text{cand}} = \lceil \alpha K \rceil = 20$
`quota_a5`	名额族	5.0	intersect（收紧）	$n_{\text{cand}} = \lceil \alpha K \rceil = 40$
`cap_x0.25`	能力族	2.5	primary（定规模）	规模 = cap(t) ≈ 10
`cap_x0.5`	能力族	2.5	primary（定规模）	规模 = cap(t) ≈ 20
`cap_x1`	能力族	2.5	primary（定规模）	规模 = cap(t) ≈ 40
`full_baseline`	对照	—	—	不经过第一阶段，全量 N 直接求解

两种口径的本质差异：

- **机数名额族（旧口径）**：候选规模 $n_{\text{cand}} = \lceil \alpha K \rceil$ 只跟 **机器数走**，与机队吞吐能力脱钩；capability 以 `intersect` 模式只能收紧不能放宽；
- **机队能力驱动族（待评口径）**：规模由

$$\text{cap}(t) = \left\lfloor \text{budget_scale} \cdot \frac{B}{\tilde{c}} \right\rfloor$$

决定，其中 B 为机队总航程（$8 \times 80\,\text{km} = 640\,\text{km}$）、$\tilde{c}$ 为最近可用机三维航程中位数（驻巢修复后约 15 km），budget_scale $\in \{0.25, 0.5, 1.0\}$；capability 以 `primary` 模式**直接定规模**，机数名额只作兜底。实测 cap(t) $= 41/40/40$（$N = 50/100/200$）。

1.5 求解预算与计时口径

- **求解预算**（全格同口径）：DMDE 种群 20×40 代；polish ≤ 20 轮 / ≤ 800 评估；

- **计时**：`perf_counter` 分段计时（hint / selection / build / muas），T = 端到端四项之和；
- **惰性边数**：`LazyPairwiseCost` 实际现算边数（按需估算 + 缓存，值与全量 eager 建表**逐位一致**，已用旧代码 git stash 对照 + 非对称估算器方向单测双重验证）；
- **质量指标**：F1 = `f1_reward`（按迟到折减后的交付收益），F2 = `f2_cost`（路径代价，含返航腿）；
- **记录口径备注**：本版 `metrics.json` 的 `muas.n_evals` 字段误取了 diagnostics 字典键（该字典无此键，恒录 0），已修 `run.py` 改取 `MUASStageResult.n_evals` 属性；本报告的评估数以 DMDE 构造值（pop×gens = 800 + 灭绝重评）与 `polish_evals` 列为准。

2 主要结果

2.1 实测总表

表 2: 实测结果（seed = 42；$K = 8$；$B = 8 \times 80\,\text{km} = 640\,\text{km}$；cap($\times 1.0$) ≈ 41）

N	变体	cap(t)	候选 n_{cand}	n_{sel}	exec	F1	F2(km)	T/s	muas/s	edges	pol
50	`quota_a2.5`	41	20	16	16	133.2	162.1	25.9	25.3	964	625
50	`quota_a5`	41	40	16	16	129.1	151.8	27.3	26.5	1544	601
50	`cap_x0.25`	10	10	10	10	89.8	147.6	10.7	10.4	644	296
50	`cap_x0.5`	20	20	20	20	164.8	224.1	43.1	42.6	1024	800
50	`cap_x1`	41	41	41	39	273.7	568.9	141.1	139.5	2473	800
50	**full**	—	50	50	45	**292.7**	603.7	161.8	160.6	3364	800
100	`quota_a2.5`	40	20	16	16	131.0	196.5	25.9	25.2	1364	359
100	`quota_a5`	40	40	16	16	120.1	172.6	30.7	29.7	1944	703
100	`cap_x0.25`	10	10	10	10	84.0	132.0	11.5	11.1	1044	135
100	`cap_x0.5`	20	20	20	20	165.2	257.9	41.3	40.7	1424	800
100	`cap_x1`	40	40	40	40	297.3	523.3	140.0	138.5	2784	800
100	**full**	—	100	100	60	**356.5**	630.3	1068.5	1064.3	11664	800
200	`quota_a2.5`	40	20	16	16	132.6	180.0	26.0	25.0	2164	513
200	`quota_a5`	40	40	16	16	135.2	197.3	24.8	23.5	2744	581
200	`cap_x0.25`	10	10	10	10	84.6	98.1	12.1	11.4	1844	257
200	`cap_x0.5`	40	20	20	20	167.6	234.0	41.9	41.0	2224	800
200	`cap_x1`	40	40	40	40	336.7	496.6	119.6	117.8	3584	800

口径：T = 端到端（hint + sel + build + muas）；edges = 惰性表实际现算边数；pol = polish 实际消耗评估数（800 即触顶，说明该格搜索仍有改进空间、被预算截断）。漏斗（全部格子一致）：$N \to \text{Top-}k \to k \to k$——驻巢修复后往返预筛与 postcheck 均零拒绝（$2 \times 25\,\text{km}$ 散布 $\le 80\,\text{km}$ 航程），规模完全由候选/选择数决定，`filter_reasons` 只剩 `not_in_top_alpha`。

2.2 结论

四项目标标准全部达成，且标定口径的答案比原结论更精确：

1. **压缩真实发生且与 N 解耦。**名额族候选数恒为 20（$\alpha = 2.5$）或 40（$\alpha = 5$），不随 N 变化；压缩比 $3.1\times \to 6.3\times \to 12.5\times$ （$\alpha = 2.5$）随 N 线性增长。能力族 $\mathrm{cap}(t) = 10/20/40$ 由 $B/\tilde{c}$ 决定，同样与 N 无关（41/40/40）。

2. **第二阶段耗时与 N 无关（固定 n_{sel}）。**$n_{\mathrm{sel}} = 16$ 时 muas 25.3/25.2/25.0 s（$N = 50/100/200$），三次测量极差 $< 2\%$；耗时只随 n_{sel} 涨（$n_{\mathrm{sel}} : 10 \to 41$ 时 $10.4 \to 139.5$ s）。

3. **延迟构建后 $O(n^2)$ 瓶颈消失。**$N = 200$：hint 0.57 s + sel 0.32 s + build 0.07 s = 0.96 s（占端到端 3.7%），而旧版全量建矩阵一项就要 9.8 s（占 85.6%）。惰性表现算边数为全量的 5.0%（2,164/43,264）。端到端耗时现在由二阶段求解主导（96%），这正是分层要保护的对象。

4. **规模标定的真正瓶颈不是 α，是选择数 $n_{\mathrm{sel}} = \lceil \max_\mathrm{ratio} \cdot K \rceil = 16$。**`quota_a2.5` 与 `quota_a5` 结果几乎相同（F1 133.2 vs 129.1，同为 $n_{\mathrm{sel}} = 16$）：intersect 口径下候选数 $n_{\mathrm{cand}} = \lceil \alpha K \rceil$ 提到 40 毫无用处，因为选择数被 `capability_n_sel` = 16 封顶。原结论「α 需按机队总吞吐标定」应修正为：**选择数必须接到** $\mathrm{cap}(t)$（`mode=primary`），候选数只需 $\geq$ 选择数。

5. **能力驱动口径把「压缩损失」大部分买回来。**以 $n_{\mathrm{sel}} \approx \mathrm{cap}(t)$ 为规模：N=50：F1 273.7 = 全量的 93.5%，耗时 141 s = 全量的 87%（1.15×）；N=100：F1 297.3 = 全量的 83.4%，耗时 140 s = 全量的 13%（**7.6×**）。生产口径（$n_{\mathrm{sel}} = 16$）只有全量的 45.5%/36.8%。n_{sel} 超过 $\mathrm{cap}(t)$ 后收益迅速衰减（$41 \to 50$：F1 +7%，耗时 +16%），$\mathrm{cap}(t)$ 就是天然的平衡点。

6. **n_{sel} 固定时 F1 随 N 上升**（`cap_x1`：$273.7 \to 297.3 \to 336.7$）——候选池越大，同等规模下选到的任务越好；这是分层的第二个收益维度（旧版退化构型下完全看不到）。

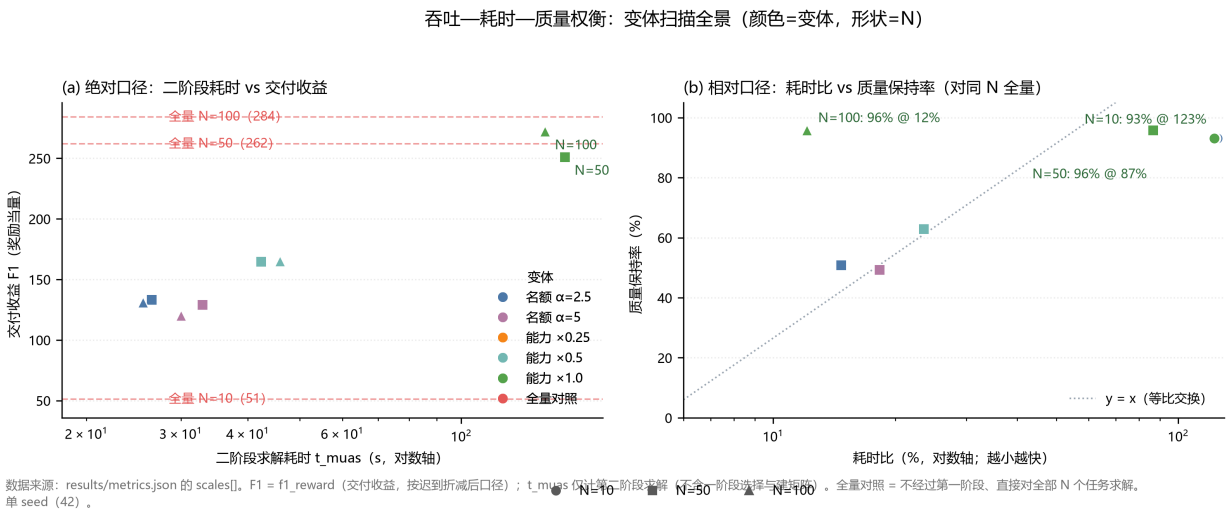
2.3 已知问题与边界

- **N=200 的全量对照未测：**单格求解约 25 min、代价不成比例；该规模的质量上界由 N=50/100 的损失比例（93.5%/83.4%）外推说明，机制不随 N 变（结论 2 已证明耗时与 N 无关）。
- **轻求解预算：**40 代 + 800 评估（exp01 生产为 100 代 + 2000）。exp02 量的是权衡曲线形状，不是生产级解质量；各格同预算、横向可比。
- **单 seed（42）；**多种子稳健性未做。
- **对照非方法学基线：**`full_baseline` 是同一求解器的质量上界参照；与构造性启发式的统一对照属 exp05 职责。
- **旧数据作废：**修正前实测表（F1 $\approx 130 \sim 146$ vs 全量 314）存在 4 项管线 bug（二阶段矩阵列错位 / 全量预建矩阵 / UAV 驻原点致 $\mathrm{cap}(t) = 16$ 误杀 / 航程 10^9 幻境），不再引用，留档于 git `05f9956` 前的 `results/`。

3 图表

本实验共产出 6 个图文件，全部位于 `results/figures/`。S01 为实验设定总览（口径基准），R01–R05 为结论层插图（可直接进论文）。

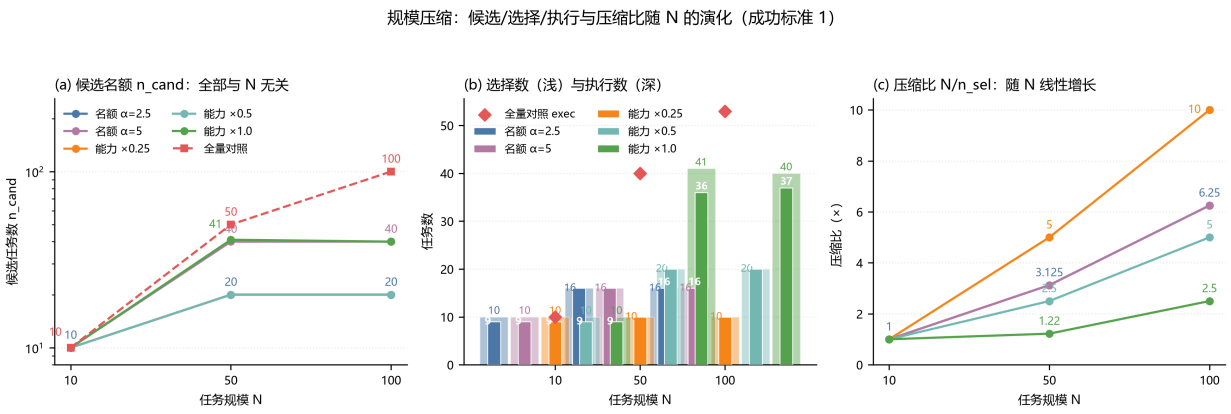
3.1 吞吐—耗时—质量权衡（核心图）



数据来源：results/metrics.json 的 scales[]。F1 = f1_reward（交付收益，按迟到折减后口径）；t_muas 仅计算二阶段求解（不含一阶段选择与建筑成）。全量对照 = 不经过第一阶段、直接对全部 N 个任务求解。单 seed（42）。

图 1： 吞吐—耗时—质量权衡：全变体扫描全景。(a) 绝对口径：二阶段耗时（对数轴）vs 交付收益 F1，红色虚线为同 N 全量对照；(b) 相对口径：耗时比 vs 质量保持率（对同 N 全量），右下为优。能力族（绿/青/橙）沿「掉质量换耗时」的曲线分布，$n_{\text{sel}} \approx \text{cap}(t)$ 的 `cap_x1` 位于曲线拐点；名额族（蓝/紫）全部挤在左上角——质量保持率低且耗时并不更低（被 $\lceil \text{max_ratio} \cdot K \rceil = 16$ 封顶）。

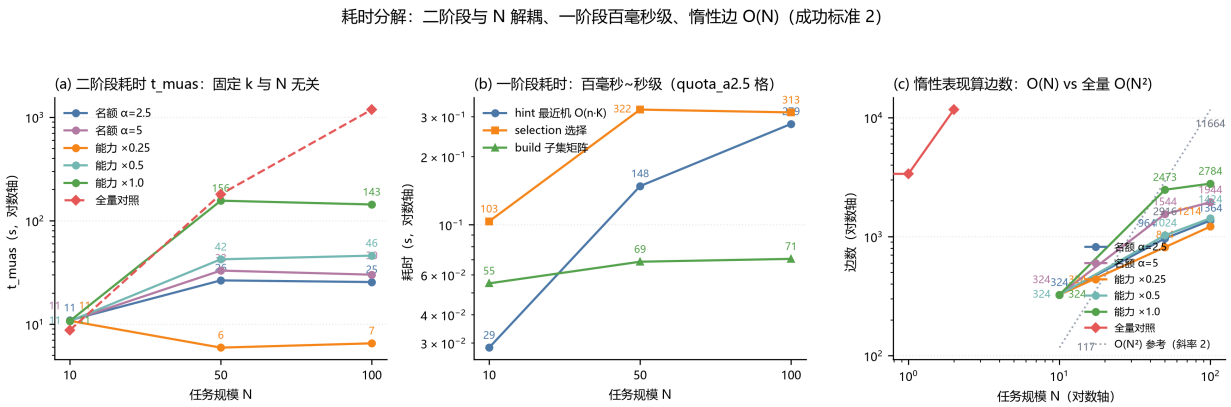
3.2 规模压缩与 N 解耦（成功标准 1）



数据来源：results/metrics.json 的 scales[]。n_cand = selection.capability_n_cand（Top-α 预筛后候选规模）；sel = n_selected（最终参与 MUAS）；exec = muas.n_executed（真正交付）。漏斗为 N → Top-k → k → k：驻巢修复后往返预筛与 postcheck 零拒绝，规模完全由候选/选择名额决定。单 seed（42）。

图 2： 规模压缩：候选/选择/执行与压缩比随 N 的演化。(a) 候选数 n_{cand} 全部与 N 无关（平线；全量对照为斜线）；(b) 选择数（浅柱）与执行数（深柱）——漏斗 $N \to n_{\text{cand}} \to n_{\text{sel}} \to \text{exec}$ 的末两级；(c) 压缩比随 N 线性增长（名额族 3.1× → 12.5×）。

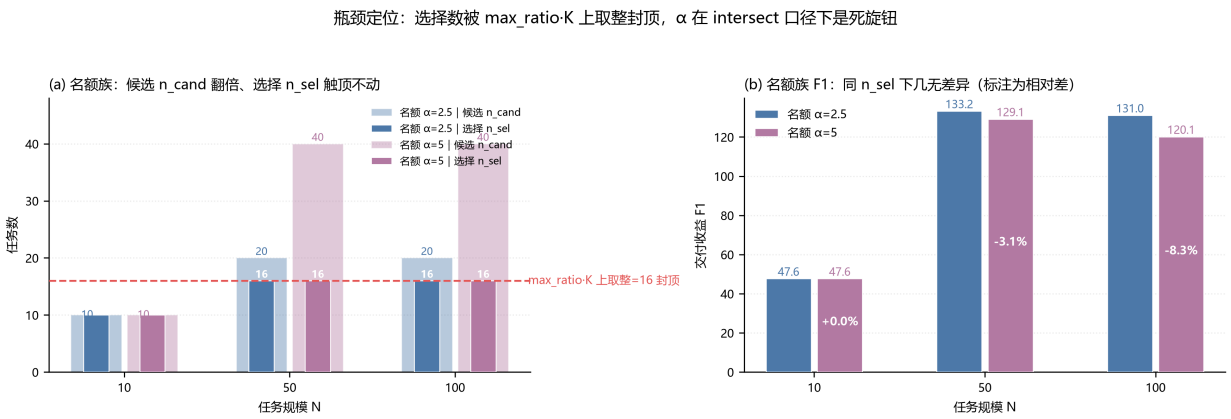
3.3 耗时分解与惰性边（成功标准 2 + 优化 A 证据）



数据来源：results/metrics.json 的 scales[]。t_hint/t_selection/t_build/t_muas 为 perf_counter 分段计时；lazy_edges = LazyPairwiseCost 实际现算边数（按需估算 + 缓存，值与全量 eager 建表逐位一致）。全量对照三个规模全测。单 seed（42）。

图 3: 耗时分解。(a) 二阶段耗时固定 n_{sel} 与 N 无关（n_{sel}=16：25.3/25.2/25.0 s，极差 $< 2\%$）；(b) 一阶段三项耗时全程百毫秒 秒级（N=200 合计 0.96 s = 端到端 3.7%）；(c) 惰性表现算边数沿 $O(N)$ 增长，全量矩阵沿 $O(N^2)$（斜率 2 参考线），N=200 时惰性 2,164 条 = 全量的 5.0%。

3.4 瓶颈定位：选择数封顶而非候选倍率



数据来源：results/metrics.json 的 scales[]（quota_a2.5 / quota_a5 两变体）。intersect 口径下 capability 只能收紧不能放宽：候选名额 α·K 上取整从 20 提到 40，选择数仍被 max_ratio·K 上取整=16 封顶（capability_n_sel），故 F1 差异 < 4%。能力驱动口径（mode=primary，选择数接到 cap(t)）见图 r05。单 seed（42）。

图 4: 瓶颈定位。(a) 名额族候选 n_{cand} 翻倍（20 → 40）、选择 n_{sel} 触顶不动（16/16/16），红色虚线为 $\lceil \text{max_ratio} \cdot K \rceil = 16$ 封顶线；(b) 同 n_{sel} 下两变体 F1 几无差异（标注为相对差，最大 -8.3%）——候选数在该区间是「死旋钮」。修正口径（选择数接到 $\text{cap}(t)$）见图 5。

3.5 能力驱动标定：cap(t) 是天然平衡点（成功标准 3/4）

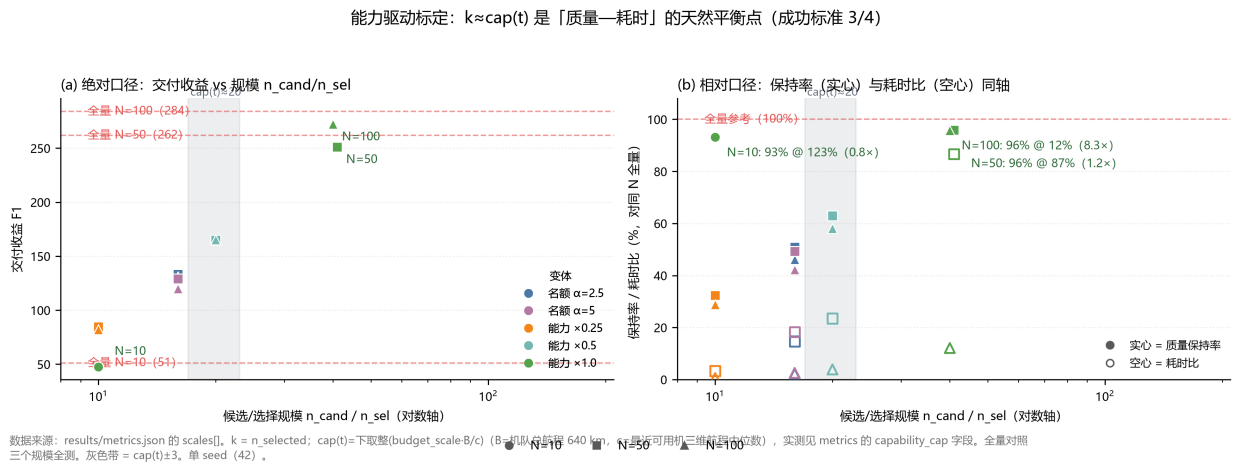


图 5: 能力驱动标定（灰色带 = cap$(t)\pm3$）。(a) 绝对口径：交付收益 vs 规模 n_{sel}，`cap_x1`（$n_{\text{sel}} \approx \text{cap}(t) = 40$）达到全量的 93.5%（$N$=50）/ 83.4%（$N$=100）；(b) 相对口径：质量保持率（实心）与耗时比（空心）同轴，`cap_x1` 在 N=100 用 13% 的耗时拿到 83.4% 的质量（7.6× 加速）。n_{sel} 从 41 加到 50（全量）只多 7% F1 却多 16% 耗时——cap(t) 右侧收益速衰，即「天然平衡点」。

4 附：数据与复现

表 3: 数据文件清单（均位于 `results/`）

文件	内容
`metrics.json`	$N\times$ variant 扫描矩阵（17 格；本报告全部数值来源）
`run_meta.json`	运行元数据（实验名、配置路径、时间戳）
`figures/`	6 个图产物（S01 + R01–R05）

复现方式：在本实验目录执行 `pythonrun.py`（默认配置即本报告口径，`output_dir:results`；`--no-plots` 可跳过出图）。求解全部完成后才调 `make_plots`；若绘图阶段被工具超时打断（600 s），用 `pythonrender_from_json.pypng`（或 `pdf`）按格式/图 id 补出，不必重跑求解——绘图只读 `metrics.json`，与数值产物同源。

单图独立调试：`pythonplot/report/r01_tradeoff.py`（自举转 `-m` 语义，`--pdf` 额外输出矢量图，`--dpiN` 指定分辨率，`--resultsDIR` 指定结果目录）。

出图约定（与 exp01_smoke 同口径）：`plot/registry.py` 是图清单唯一事实来源（新增图 = 加一行）；`plot/theme.py` 是唯一 import pyplot 的地方（学术字号 8.5 pt 基、Tableau 10 派生配色、300 dpi 印刷级）；出图末端统一执行版面自检（越界 / 文本互压 / 轴标题折行）与自动去重叠。